Khulna University of Engineering & Technology
Department of Computer Science and Engineering

Computer Graphics / Image Processing Lab

Jungle Rest Stop

An interactive 3D scene built only from basic shapes
with OpenGL 3.3



Student: Md. Nayeem
Roll: 2107050
Language / API: C++17, OpenGL 3.3 Core, GLFW, GLAD
Date: October 2026

Abstract

This report presents *Jungle Rest Stop*, a real-time 3D scene made with OpenGL 3.3. A small rest stop stands in a clearing in a thick jungle. It has three connected rooms: a main lounge with sofas, a TV, a coffee machine and three working vending machines; an ATM room where the ATM shows every step of a withdrawal; and a library with an aquarium. Visitors are computer-controlled people (NPCs). They arrive only by car, park, walk inside, use the machines, sit, watch TV or read, and then drive away. The user controls one person and can view the scene in first person, third person or with a free camera, and can change the field of view.

Every object is made only from basic shapes (cube, cylinder, sphere, cone, torus, wedge and a few more) using translation, rotation and scaling. No 3D model files are loaded, and no texture images are loaded either: all textures are generated in code. A single key (or an on-screen button) switches between a *Basic* mode (flat colours with simple lighting) and an *Enhanced* mode (textures, shadow mapping, fog, sky, wind and tone mapping). The scene also has a full day and night cycle and many kinds of light. Because the target computer has only an integrated GPU (Intel Iris Xe), the program uses instanced drawing, chunk culling and per-room light lists, and it keeps the frame rate capped at a steady 60 (or 30) frames per second.

Contents

1 Introduction **3**

2 Objectives **3**

3 Tools and Environment **3**

4 From the Old Project to the New One **4**

5 System Design **4**
5.1 Modules 4
5.2 One frame of the program 5
5.3 Scene layout 6

6 The Basic Shapes **6**

7 Transformations and Hierarchical Modelling **8**
7.1 The three basic transformations 8
7.2 Order of operations 8
7.3 The `Node` helper 8
7.4 Example hierarchy: the human body 9

8 Building Every Object from Basic Shapes **9**
8.1 Vending machine 9
8.2 Products 11
8.3 ATM 12
8.4 Coffee machine, TV and aquarium 12
8.5 Furniture and lamps 13
8.6 People 14
8.7 Cars 15
8.8 The building 16
8.9 The jungle 17

9 Lighting **18**
9.1 Lighting model 18
9.2 Light types 19
9.3 Choosing lights every frame 19

10 Day and Night **20**

11 Textures **21**
11.1 Procedural textures 21
11.2 How textures are applied 22

12 Shadows **23**

13 Sky, Fog, Tone Mapping and Wind **24**

14 Basic Mode versus Enhanced Mode **24**

15 Interaction: Using the Machines **25**
15.1 Look-and-press picking 25
15.2 ATM: step by step 26
15.3 Coffee machine and TV 27

16 NPC Visitors **27**
16.1 Life of a visitor 27
16.2 Driving and parking 27
16.3 Walking and activities 28

17 The Player and the Cameras **29**

18 Performance on a Weak GPU **30**
18.1 Techniques 30
18.2 Measured speed 30

19 Controls **31**

20 More Screenshots **32**

21 Challenges and Solutions **33**

22 Conclusion **33**

1 Introduction

Computer graphics lets us build a whole world inside the computer and walk through it. In this lab project the goal was to upgrade an older, simple vending machine scene into a complete, living place, while still following the main rule of the course: *build everything from basic shapes by using transformations.*

The final program shows:

- a building with **three connected rooms** (main lounge, ATM room, library),
- **three fully working vending machines**. You insert money, type a code, the spiral turns, the item falls, and you take it out through the flap and collect your change,
- an **ATM** that guides the user step by step, both on its own screen and in a side panel,
- a **coffee machine**, a **TV** with four animated channels and an **aquarium** with fish,
- a parking lot and a road through a **procedural jungle** (trees, palms, bamboo, banana plants, ferns, grass, flowers, rocks, logs, vines, birds and fireflies),
- **NPC visitors** who come only by car and use the building like real people,
- **day and night**, light bulbs, lamps, spot lights, street lights and car headlights,
- a **Basic / Enhanced** switch, first person, third person and free cameras, and **FOV** control.

2 Objectives

1. Study the old project and find its weak points.
2. Model every object only with basic shapes and 3D transformations (translate, rotate, scale), using hierarchical (parent–child) modelling.
3. Build three connected rooms with realistic furniture and working machines.
4. Make NPC visitors who arrive by car, park, enter, act and leave.
5. Show realistic lighting: several light types, real bulb models, and a day and night cycle.
6. Add textures, shadows, fog and a sky, with a switch back to a basic look.
7. Make the outside a dense jungle.
8. Keep the code modular, and keep the frame rate between 30 and 60 FPS on a weak GPU.
9. Write a clear report with screenshots.

3 Tools and Environment

Item	Used
Language	C++17 (MinGW-w64 / MSYS2 `g++`)
Graphics API	OpenGL 3.3 Core profile, GLSL 330
Window / input	GLFW 3
OpenGL loader	GLAD 2
Text	Windows GDI renders one font atlas at start-up (Consolas)
Assets	**none**: all meshes and all textures are generated in code
Test PC	Intel Core i5-1135G7, **Intel Iris Xe (integrated GPU)**, 1280×720 window, 4×MSAA

4 From the Old Project to the New One

The old project (kept in the `legacy/` folder) was one file, `main.cpp`, with 2,341 lines. It drew each shape with its own draw call and looked up every uniform by name for every shape. Its trees were a few spheres on cylinders, and it had no textures, shadows or text on screens. Table 1 compares the old and new versions.

Table 1: Old project versus the upgraded project

Topic	Old project	New project
Code structure	1 big file + 4 headers	46 files in 8 modules (about 9,400 lines)
Drawing	1 draw call per shape, uniform lookup each time	instanced drawing; static world baked into 101 chunks
Shapes	cube, cylinder, sphere, coil, plane	15 basic shapes + terrain grid
Textures	none	25 procedural textures (texture array, mip-maps)
Shadows / fog / sky	no / no / flat colour	shadow map with PCF / distance fog / sky with clouds, sun, moon, stars
Rooms	vending, ATM, ticket rooms (boxes)	lounge, ATM room, library with windows, doors and real furniture
Machines	keys print a message	full state machines, screens with text, coins, cards, cash, receipts
People	one avatar	articulated humans with walk, sit, drive, press and bend poses + NPC AI
Cars	none	sedan, SUV and hatchback with steering, spinning wheels, doors and lights
Outside	31 trees made of spheres	797 rainforest trees, 296 palms, bamboo, banana plants, 2,232 ferns, 4,758 grass tufts
Time of day	fixed	24 h cycle, automatic street lamps
Cameras	orbit + first person	first person, third person, free camera + FOV
Frame rate	sleep to 100 FPS	accurate cap at 60 or 30 FPS

5 System Design

5.1 Modules

The code is split into small modules. Each one has a single job (Figure 1).

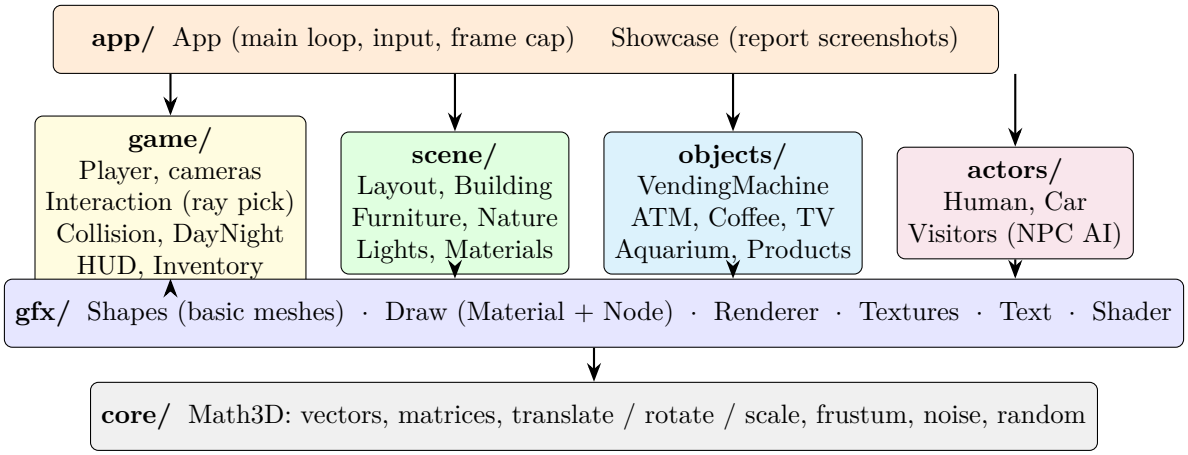


Figure 1: Module overview. Arrows mean "uses".

5.2 One frame of the program

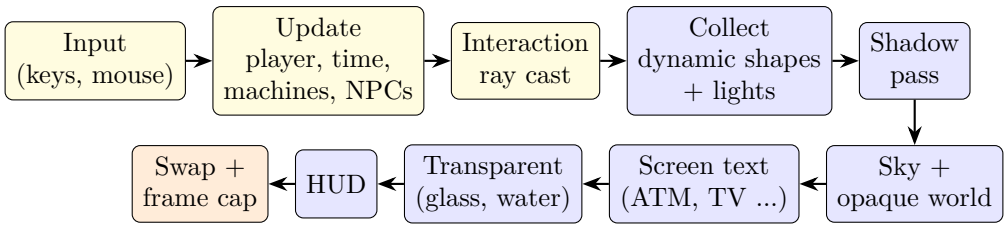


Figure 2: Steps of one frame.

5.3 Scene layout

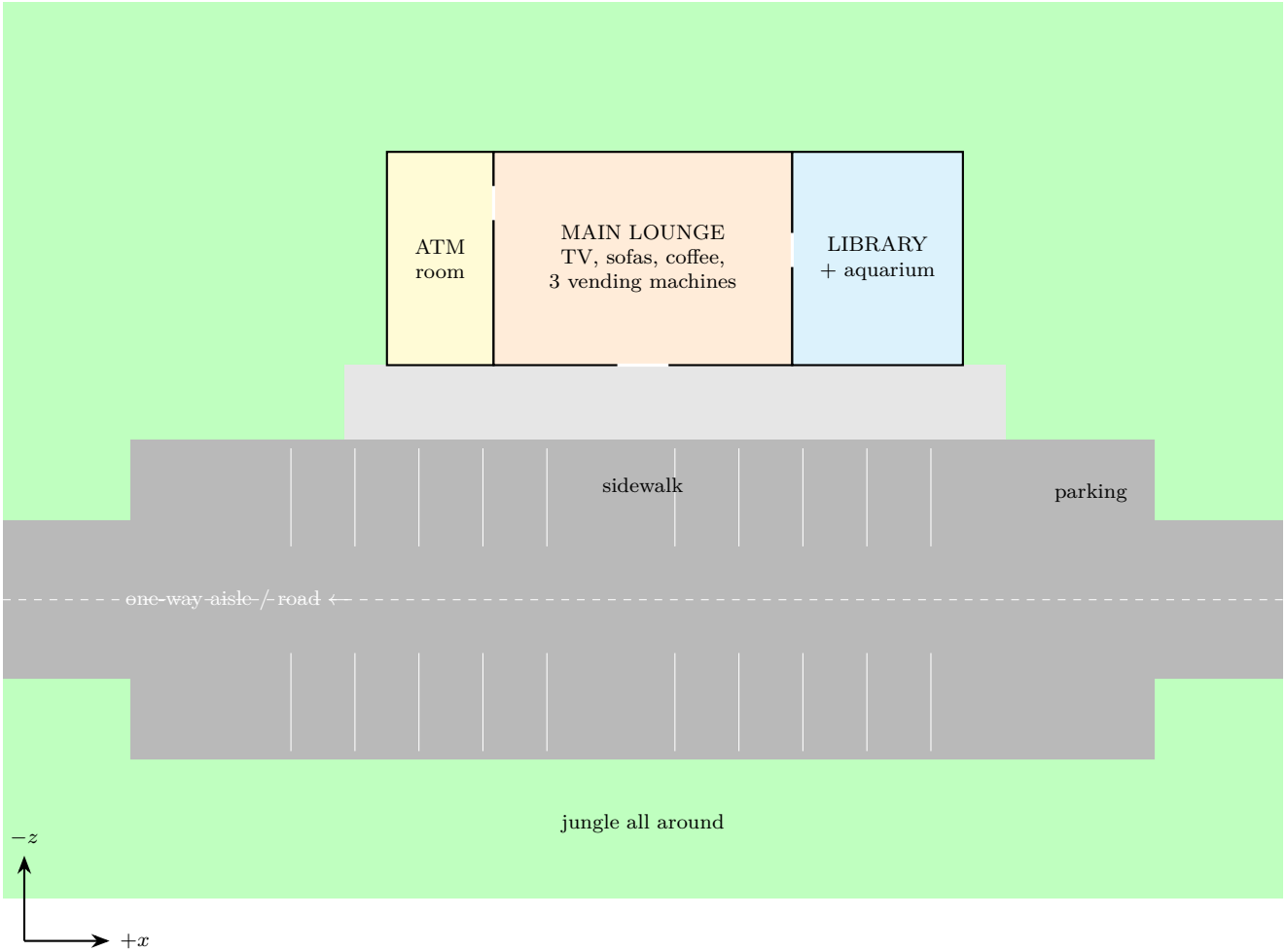


Figure 3: Top view of the layout (1 grid unit = 1 m). The building faces $+z$, towards the parking lot.

6 The Basic Shapes

Everything is built from the small set of *unit* meshes in Table 2. Each one is centred at the origin with a size of about one unit. They are created once at start-up (`gfx/Shapes.cpp`) and kept on the GPU.

Table 2: Basic shapes and how they are generated

Shape	How it is generated	Typical use
Cube	6 faces × 4 vertices (flat normals), $[-0.5, 0.5]^3$	walls, panels, keys, books
Cylinder	ring of 24 vertices at $y = \pm 0.5$, radius 0.5, plus two caps	legs, poles, cans, wheels
Cylinder (low)	same, 8 sides only	tree trunks, bamboo
Sphere	latitude/longitude grid, 14 × 20	heads, LEDs, bulbs
Sphere (low)	5 × 8 grid	foliage blobs, rocks
Cone	cylinder with top radius 0	fish tails, snake-plant leaves
Truncated cone	cylinder with top radius 0.35	cups, pots, lamp shades, poles
Torus	ring radius 0.4, tube radius 0.1	clock rim, tyres on rings, collars
Hemisphere	top half of a sphere + bottom disc	hair, domes, bowl
Wedge	right-angled triangle extruded along x	bonnet, buttress roots, fins
Tapered box	bottom 1 × 1, top 0.7 × 0.6	car cabin, chest, lamp head
Quad	one square in the xy plane	palm fronds, vines
Cross card	two crossed quads, $y \in [0, 1]$	grass, ferns, flowers
Leaf card	three crossed quads	leaf clusters of trees
Coil	tube swept along a helix (5 turns)	vending machine spirals
Terrain grid	150 × 150 grid displaced by $h(x, z)$	the ground

The curved shapes come from simple parametric formulas. For a cylinder (radius r, angle $\theta_i = 2\pi i/n$):

$$\mathbf{p}_i^{top} = (r\cos\theta_i,\ 0.5,\ r\sin\theta_i), \qquad \mathbf{p}_i^{bottom} = (r\cos\theta_i,\ -0.5,\ r\sin\theta_i), \qquad \mathbf{n}_i = (\cos\theta_i,\ 0,\ \sin\theta_i).$$

A sphere uses $\mathbf{p}(\phi, \theta) = 0.5\,(\sin\phi\cos\theta,\ \cos\phi,\ \sin\phi\sin\theta)$ with normal $\mathbf{n} = 2\mathbf{p}$. A torus uses $\mathbf{p}(u, w) = (R + r\cos w)(\cos u, 0, \sin u) + (0, r\sin w, 0)$. The coil follows the helix $\mathbf{c}(t) = (R\cos 2\pi kt,\ R\sin 2\pi kt,\ L(t - 0.5))$, and a small circle is placed around each point of it.

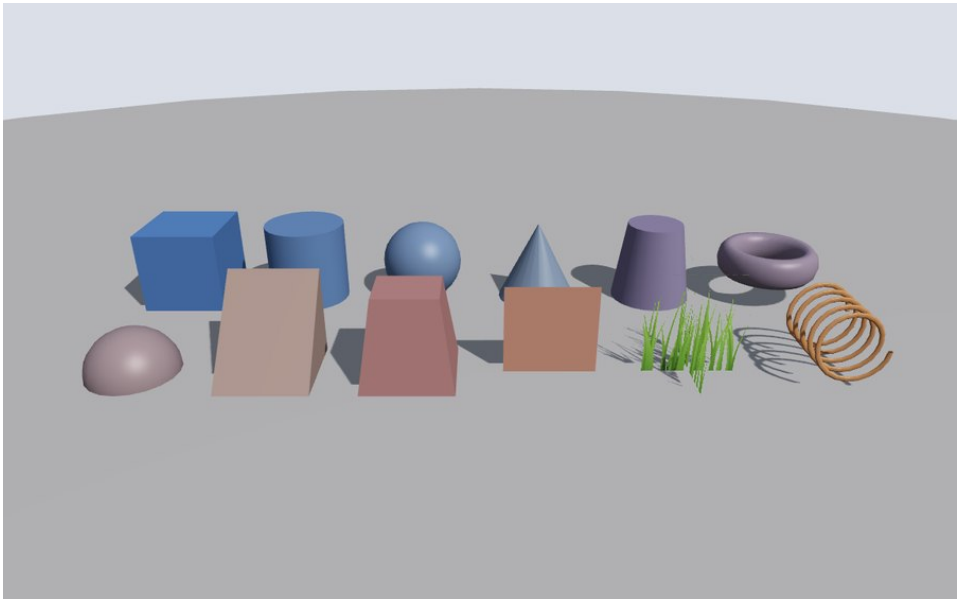


Figure 4: The basic shapes rendered by the program (cube, cylinder, sphere, cone, truncated cone, torus, hemisphere, wedge, tapered box, quad, cross card, coil).

7 Transformations and Hierarchical Modelling

7.1 The three basic transformations

All matrices are 4×4 and column-major, so they can go straight to OpenGL.

$$T(t) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad S(s) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

R_x and R_z work in the same way. Rotation about any axis $\mathbf{a}$ uses Rodrigues' formula $R = \cos\theta\, I + (1 - \cos\theta)\, \mathbf{a}\mathbf{a}^T + \sin\theta\, [\mathbf{a}]_\times$. The camera uses the usual `lookAt` view matrix and a perspective projection

$$P = \begin{bmatrix} \frac{1}{a\tan(f/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(f/2)} & 0 & 0 \\ 0 & 0 & -\frac{z_f + z_n}{z_f - z_n} & -\frac{2 z_f z_n}{z_f - z_n} \\ 0 & 0 & -1 & 0 \end{bmatrix},$$

where f is the field of view, which the user changes with the mouse wheel (see Section 17).

7.2 Order of operations

Every part of every object is placed with

$$M_{part} = M_{parent} \cdot T(\text{position}) \cdot R(\text{rotation}) \cdot S(\text{size}).$$

The unit shape is first **scaled** to the right size, then **rotated**, then **moved** to its place inside the parent, and finally the parent's own matrix carries it into the world. Normals use the normal matrix $N = (M^{-1})^T$ (upper 3×3 part), which keeps lighting correct even when a shape is stretched.

7.3 The `Node` helper

Objects are written as trees of `Node`s. A child node only stores the extra transform relative to its parent, so moving the parent moves all its children too.

```
struct Node {
    DrawSink* sink; Mat4 M; // M = this node's world matrix
    Node at(const Vec3& p, const Vec3& rotDeg) const // child: M * T(p) * R(rot)
        { return Node(*sink, rotateYXZ(translate(M, p), rotDeg)); }
    Node rotX(float deg) const { return rot(deg, Vec3(1,0,0)); }
    void part(MeshId mesh, Vec3 pos, Vec3 size, const Material& m, Vec3 rotDeg = {}) const
        { sink->add(mesh, scale(rotateYXZ(translate(M, pos), rotDeg), size), m); }
};
// Example: a sofa arm = a box plus a "rolled" cylinder lying along Z
n.box({x, 0.42f, 0.0f}, {0.16f, 0.44f, 0.88f}, fabric);
n.part(M_CYLINDER, {x, 0.64f, 0.0f}, {0.19f, 0.88f, 0.19f}, fabric, {90, 0, 0});
```

The same object code can send its shapes to different "sinks": the static world, the per-frame list, a counter (used for the tables in this report) or a limiter that only lets the first k parts through. The limiter is how the "build stage" pictures below were made.

7.4 Example hierarchy: the human body

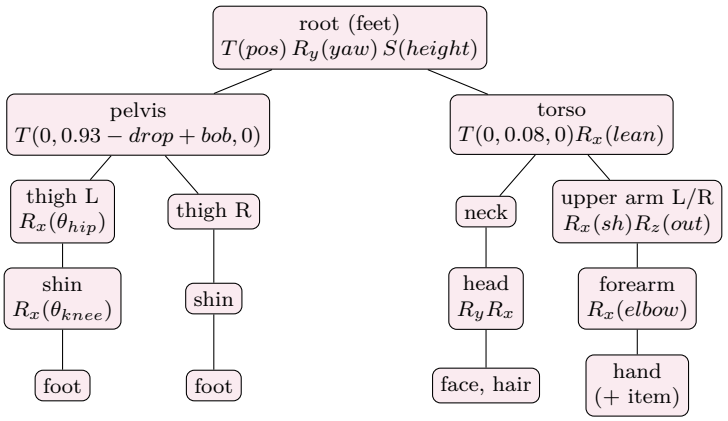


Figure 5: Scene graph of a person. The pose only changes the joint angles θ, while every body part stays attached to its parent.

8 Building Every Object from Basic Shapes

Table 3 lists how many basic shapes each object uses and which ones. The numbers are counted by the program itself (a counting sink). After the table, each important object is explained with its parts and transformations. The *build stage* strips show the object while it is being assembled, after 12%, 35% and 65% of its parts and when it is complete.

Table 3: Number of basic shapes per object (Cyl-L / Sph-L = low-poly versions, TCone = truncated cone, TBox = tapered box)

Object	Total	Cube	Cyl	Cyl-L	Sph	Sph-L	Cone	TCone	Torus	Hemi	Wedge	TBox	Quad	Cross	Leaf	Coil
Vending machine	**287**	79	177	–	3	–	–	16	–	–	–	–	–	–	–	12
ATM	**49**	48	–	–	–	–	–	–	–	1	–	–	–	–	–	–
Coffee machine	**32**	16	9	–	1	–	–	2	4	–	–	–	–	–	–	–
Television (frame)	**5**	4	–	–	1	–	–	–	–	–	–	–	–	–	–	–
Sofa (3 seats)	**21**	12	9	–	–	–	–	–	–	–	–	–	–	–	–	–
Armchair	**13**	6	7	–	–	–	–	–	–	–	–	–	–	–	–	–
Bookshelf	**21**	21	–	–	–	–	–	–	–	–	–	–	–	–	–	–
Person (standing)	**41**	9	11	–	17	–	–	1	1	1	–	1	–	–	–	–
Person (walking)	**40**	9	11	–	17	–	–	–	1	1	–	1	–	–	–	–
Person + chair	**47**	15	11	–	18	–	–	–	1	1	–	1	–	–	–	–
Sedan car + driver	**115**	63	28	–	18	–	–	–	2	1	1	2	–	–	–	–
SUV + driver	**117**	66	28	–	18	–	–	–	2	1	1	1	–	–	–	–
Hatchback + driver	**114**	62	28	–	18	–	–	–	2	1	1	2	–	–	–	–
Rainforest tree	**27**	–	–	5	–	6	–	–	–	–	4	–	–	–	12	–
Coconut palm	**37**	–	–	14	–	5	–	–	–	–	–	–	18	–	–	–
Bamboo clump	**130**	–	–	100	–	–	–	–	–	–	–	–	–	–	30	–
Banana plant	**28**	–	–	6	–	22	–	–	–	–	–	–	–	–	–	–
Street lamp	**5**	1	2	–	–	–	–	1	–	–	–	1	–	–	–	–
Pendant lamp	**5**	–	3	–	1	–	–	1	–	–	–	–	–	–	–	–
Aquarium	**76**	12	–	1	35	6	7	1	–	–	7	–	–	7	–	–
6 products	**23**	9	12	–	1	–	–	1	–	–	–	–	–	–	–	–
Table set	**23**	10	2	–	3	1	–	1	–	1	–	–	–	–	5	–
Wall clock	**17**	14	2	–	–	–	–	–	1	–	–	–	–	–	–	–
Bush+fern+rock+log	**23**	–	–	4	–	8	–	–	–	3	–	–	5	1	2	–

8.1 Vending machine

The machine is $1.0 \times 1.95 \times 0.8$ m. Its origin is at the floor in the middle, and it faces $+z$. Its *cabinet* is drawn once into the static world. The moving parts (spirals, products, keys, LEDs, flap and coins) are drawn every frame.

Table 4: Main parts of the vending machine (sizes and positions in metres, in the machine's own space)

Part	Shape	Size (s_x, s_y, s_z)	Position / rotation
Back, sides, top, base	cube	back $(1, 1.86, .04)$, side $(.04, 1.86, .8)$	back $(0, 1.0, -.38)$, sides $(\pm.48, 1.0, 0)$, top $(0, 1.93, 0)$
4 levelling feet	cylinder	$(.06, .05, .06)$	$(\pm.42, .025, \pm.32)$
Lit marquee (title)	cube, emissive	$(.94, .12, .02)$	$(0, 1.86, .39)$
Display cavity + tube light	cube, cylinder	tube $(.022, .6, .022)$	tube rotated $R_z(90°)$ at $(-.15, 1.76, .30)$
4 shelves A–D	cube	$(.62, .015, .38)$	$y_r = 1.555,\ 1.285,\ 1.015,\ 0.745$
12 spirals	coil	radius .055, 5 turns, .30 long	$(x_c, y_r + .065, -.06)$, $R_z(-spin)$
48 products	see Table 5		4 per slot, $z = .0525 - .075k$
Front glass	cube, $\alpha = 0.18$	$(.62, 1.14, .008)$	$(-.15, 1.22, .385)$
Control panel + LCD	cube	$(.27, 1.30, .04)$	$(.33, 1.20, .38)$
9 keys (A B C D 1 2 3 CLR OK)	cube	$(.05, .045, .014)$	grid $x = .255/.325/.395$, $y = 1.50 - .06\,row$; pushed in by .007 m when pressed
3 status LEDs	sphere, emissive	.022	ready (green), vending (amber), item (blue)
Coin slot, note slot, card pad, coin cup	cube	small plates	$y = 1.23,\ 1.12,\ 1.00,\ 0.86$
Coin-return lever	cube	$(.02, .05, .02)$	$R_x(40° \cdot press)$
Collection flap	cube (child node)	$(.58, .34, .015)$	hinge node $T(-.15, .57, .39) \cdot R_x(\theta_{flap})$
Returned coins	cylinder	$(.024, .004, .024)$	fall with gravity into the cup

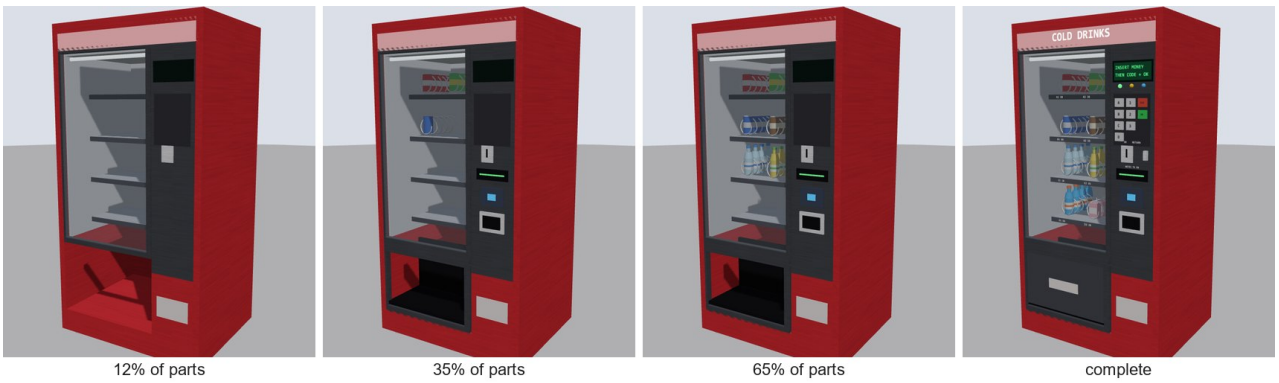


Figure 6: Vending machine build stages.

How it works. Each machine is a small *state machine* (Figure 7). Money is kept as an integer number of Taka, so there are no rounding errors. When you press **OK**, the spiral turns 360° in 1.6 s, and the products move forward by one place (0.075 m). The front product then becomes a free body with velocity $\mathbf{v}$, gravity $g = 9.81\,\mathrm{m/s^2}$ and spin. When it hits the tray floor it bounces with restitution 0.32:

$$\mathbf{v}_y \leftarrow \mathbf{v}_y - g\,\Delta t, \qquad \mathbf{p} \leftarrow \mathbf{p} + \mathbf{v}\,\Delta t, \qquad \text{if } p_y < y_{floor}:\ v_y \leftarrow -0.32\,v_y.$$

When the bounce speed drops below 0.35 m/s, the product lies on its side and waits. Pushing the flap rotates it inward around its top hinge, and the item goes into the player's bag. Any change falls as coins into the coin cup. An empty slot is refilled automatically after 40 s.

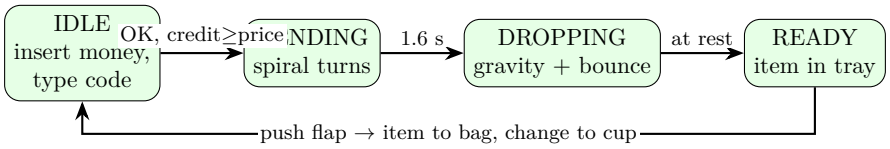


Figure 7: Vending machine state machine.



(a) Code B1 typed, credit enough **(b)** Spiral pushes the can out **(c)** "Take your item"

Figure 8: Buying an energy drink. The LCD, the price tags and the key labels are real text drawn on the machine.

8.2 Products

Table 5: Products and their shapes

Product	Built from
Can	cylinder body $(.064, .11, .064)$ + colour band cylinder + 2 thin metal rim cylinders
Bottle	cylinder body + truncated-cone shoulder + thin neck cylinder + cap cylinder + label cylinders
Chips bag	box body + flattened sphere "puff" + 2 crimp boxes + round logo (cylinder rotated $R_x 90°$)
Chocolate bar	box wrapper + narrower box band
Cookie box	box + stripe box + round cookie picture (cylinder)
Juice box	box carton + front label box + tilted straw cylinder

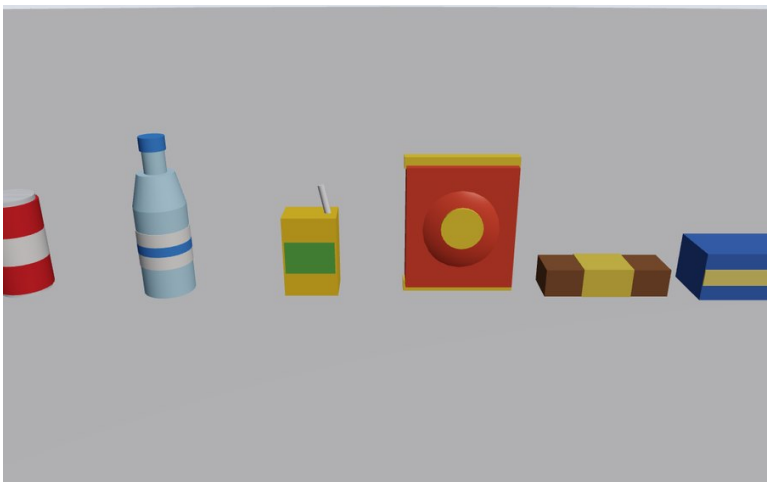


Figure 9: Products: can, bottle, juice box, chips bag, chocolate bar, cookie box.

8.3 ATM

Table 6: Main parts of the ATM

Part	Shape	Size / transformation
Plinth, lower cabinet, upper fascia	cubes	$(1.0, .08, .72)$, $(.96, .92, .66)$, $(.96, .80, .50)$
Header with light strip	cubes	$(1.04, .24, .72)$ at $y = 1.96$; emissive strip under it
Privacy wings	cubes	$(.04, .62, .30)$ at $x = \pm 0.5$
Keypad shelf	cube in a child node	node $T(0, 1.035, 0.40) \cdot R_x(12°)$ so the keypad tilts towards the user
16 keypad keys	cubes	$(.062, .018, .042)$ on the tilted node; CANCEL red, CLEAR yellow, ENTER green
Screen bezel + screen	cube + text canvas	0.40×0.30 m screen drawn with rectangles and text
8 side keys	cubes	$(.04, .03, .016)$ at $x = \pm .245$, four rows
Card, cash, receipt	cubes	card slides along z, 4 bills slide out, receipt rotates $R_x(25°a)$
Camera dome	hemisphere	rotated $R_x(90°)$ above the screen

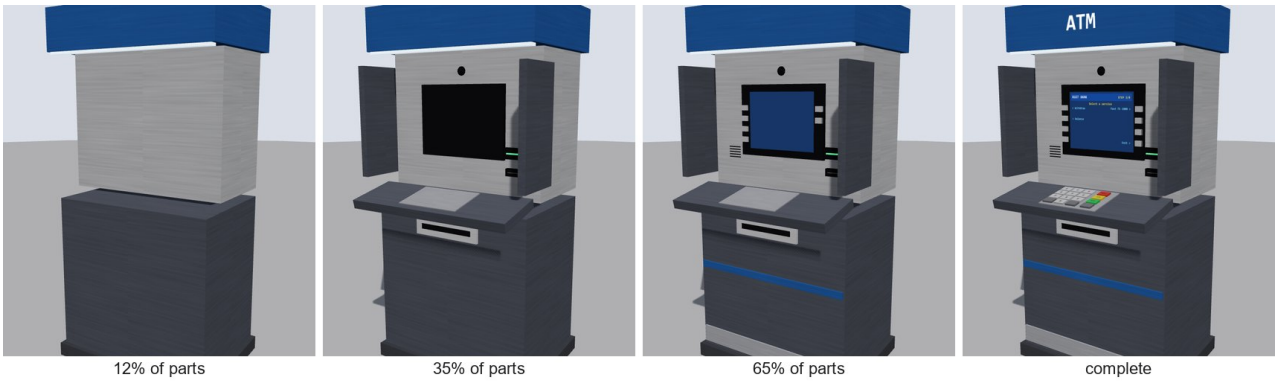


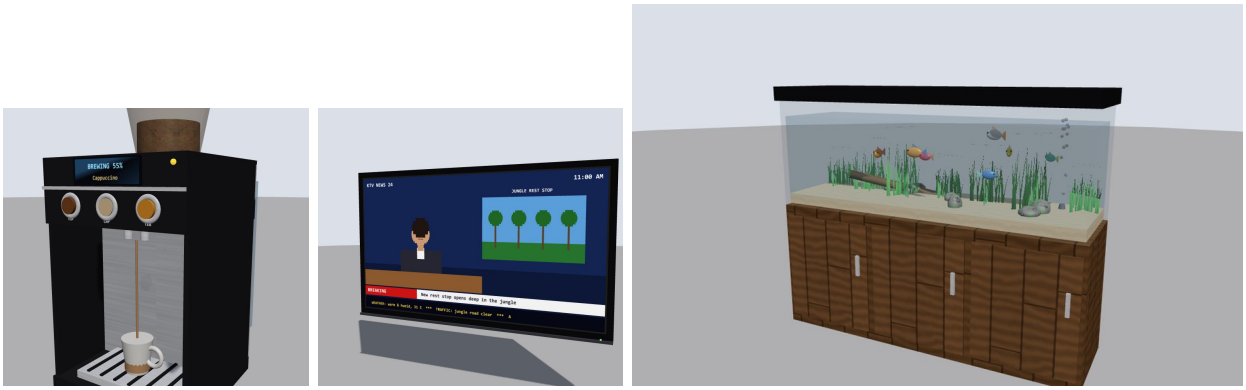
Figure 10: ATM build stages.

8.4 Coffee machine, TV and aquarium

Coffee machine. It has a base box, a steel column, two black side panels, a head box, two nozzle cylinders, a drip tray (box + 6 thin bars), a bean hopper (a glass truncated cone turned upside down with $R_x(180°)$, so it is wider at the top), a beans cylinder, a lid, a glass water tank, three round buttons (cylinders rotated $R_x(90°)$ with torus rings) and an LCD. The cup is an upside-down truncated cone with a torus handle. The coffee inside is a cylinder whose height is $0.07 \cdot fill$. The pouring stream is a thin cylinder that reaches from the nozzle to the coffee surface. Steam is made of small transparent spheres that rise, grow and fade.

TV. It has a wall bracket, a black body box, an emissive screen and a power LED. The four channels (news, nature, sports, weather) are 2D drawings made from rectangles and text on a canvas laid on the screen. Circles (heads, sun, trees, ball) are drawn as horizontal strips of width $2\sqrt{r^2 - y^2}$. When the TV is on, it also adds a flickering coloured point light to the room.

Aquarium. It has a wooden stand (box + 3 doors + handles), a sand box, rock spheres, a driftwood cylinder, water plants (cross cards that sway), a hood and a light strip. The water and glass are transparent boxes. Each fish is a scaled sphere body $(0.45s, 0.75s, 1.25s)$, a cone tail that wiggles with $R_y(25° \sin 9t)$, a wedge fin and two sphere eyes. Fish swim on ellipses $x = r_x \cos\varphi$, $z = r_z \sin\varphi$, and face their direction of motion, $yaw = \text{atan2}(\Delta x, \Delta z)$. "Feed the fish" drops food and the fish rise to it.



(a) Coffee machine **(b)** TV **(c)** Aquarium

Figure 11: Appliances, rendered alone in a "studio".

8.5 Furniture and lamps

Table 7: Furniture made from basic shapes

Object	Construction
Sofa (n seats)	width $W = 0.72n + 0.32$; base box, back box; per seat: cushion box + front roll cylinder ($R_z 90°$) + back cushion box tilted $R_x(-10°)$; two arms (box + rolled cylinder $R_x 90°$); 4 wooden leg cylinders; throw pillows (boxes rotated in 3 axes)
Armchair	the same function with $n = 1$
Coffee table	top and shelf boxes, 4 metal leg boxes, magazines (thin boxes), fruit bowl (hemisphere with negative y scale) + spheres
Bookshelf	2 side boards, top, back, plinth, 5 shelves; books are boxes with the procedural *books* texture (mesh UVs). Each shelf uses two blocks of different widths, one turned 180°, so the pattern does not repeat
Wall clock	torus rim ($R_x 90°$), face cylinder, 12 ticks (each in a node $R_z(-30°i)$), hands $R_z(-30°h)$ and $R_z(-6°m)$. The hands show the *in-game* time
Pendant lamp	ceiling disc, cord cylinder, shade (truncated cone), diffuser disc, bulb sphere (emissive when on)
Street lamp	concrete base cylinder, tapered pole (truncated cone), arm cylinder $R_x 90°$, head (tapered box $R_x 180°$), glowing lens
Others	table lamps, floor lamp, potted plants (3 kinds), counters with cabinets, water cooler, benches, bins, stanchions with ropes, security cameras (they pan), smoke detectors, roof AC units, sign boards

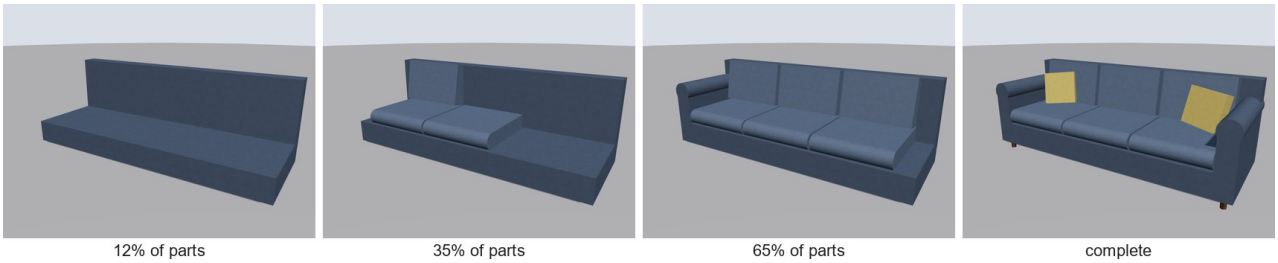
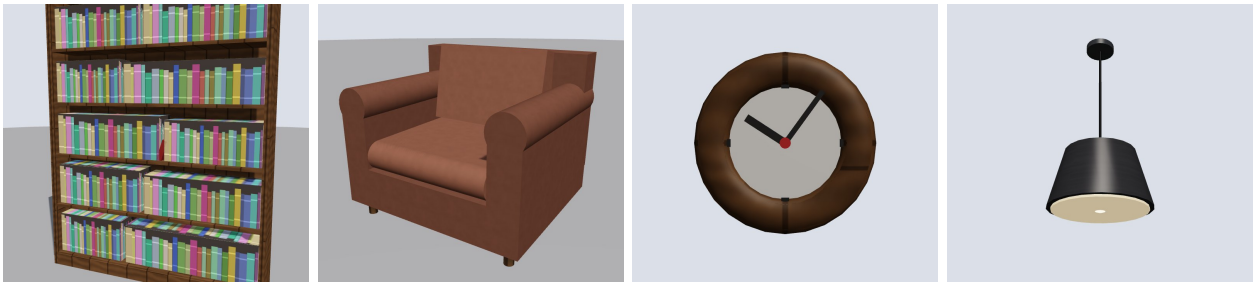


Figure 12: Sofa build stages.



(a) Bookshelf **(b)** Armchair **(c)** Wall clock **(d)** Pendant lamp

Figure 13: Furniture and lamps.

8.6 People

A person has 41 parts (Figure 5). Different looks come from a random seed: skin, hair colour and style (short, long, bun, cap), shirt, trousers, shoes, height (0.92–1.06), body width, long or short sleeves, glasses, and a long kurta-style skirt.

Table 8: Body parts of a person (local sizes in metres, w = width factor)

Part	Shape	Size	Placement
Pelvis + belt	cylinder, cube	$(.34w, .18, .21)$	pelvis node at $y = 0.93 - drop + bob$
Thigh, knee, shin	cylinder, sphere, cylinder	$(.15w, .45, .16)$, $(.12w, .42, .13)$	hip joint $(\pm.095w, -.03, 0)$, $R_x(\theta_{thigh})$; knee joint $T(0, -.45, 0)R_x(\theta_{knee})$
Shoe + sole	cubes	$(.11, .07, .26)$	ankle node, toes point $+z$
Abdomen, chest	cylinder, sphere	$(.31w, .26, .20)$, $(.40w, .36, .25)$	torso node $T(0, .08, 0)R_x(lean)$
Upper chest	tapered box $R_z(180°)$	$(.38w, .18, .22)$	upside down, so it is wide at the shoulders
Shoulders, collar	spheres, torus		$(\pm.19w, .42, 0)$
Head	sphere	$(.20, .245, .215)$	neck node, then $R_y(\text{headYaw})R_x(\text{headPitch})$
Face	spheres + boxes		eyes (white + pupil), nose, ears, eyebrows, mouth; glasses = 2 tori
Hair	hemisphere (+ sphere / box)		4 styles
Upper arm, forearm, hand	cylinder, cylinder, sphere	$(.095, .29, .095)$, $(.08, .26, .08)$	shoulder $R_x(\theta_s)R_z(\pm out)$, elbow $R_x(\theta_e)$; a held item (can, cup, snack, phone) is a child of the right hand

Animation. Only the joint angles change. For walking with phase φ:

$$\theta^{L,R}_{thigh} = \mp 26° \sin\varphi, \quad \theta^{L}_{knee} = 38° \max(0, \cos\varphi) + 4°, \quad \theta^{L,R}_{shoulder} = \pm 22° \sin\varphi, \quad bob = 0.035\,(1 - |\sin\varphi|).$$

Sitting lowers the pelvis by 0.44 m, turns the thighs $-88°$ and bends the knees $88°$. Driving, pressing buttons and bending down are other fixed poses. Changes between poses are smooth, because every angle is blended linearly over 0.35 s.

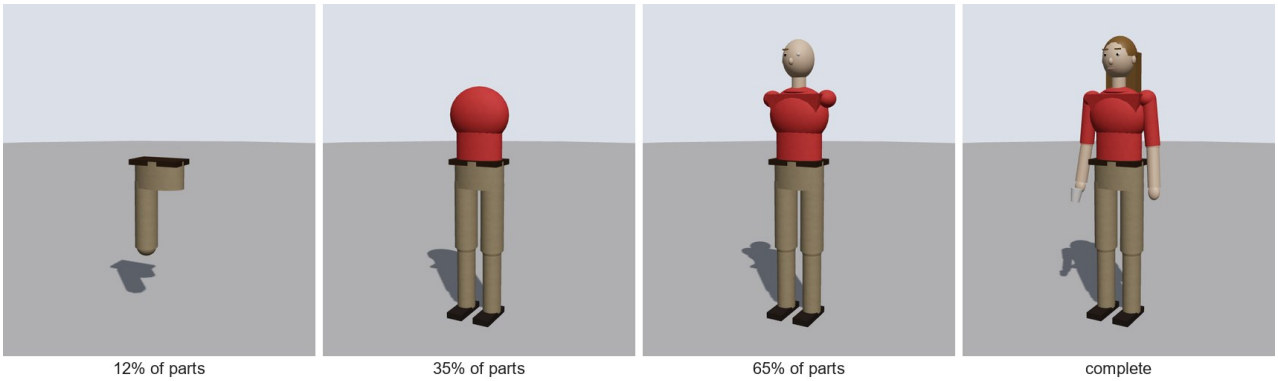


Figure 14: Person build stages.



(a) Standing, holding a cup **(b)** Walking pose **(c)** Sitting on a chair

Figure 15: Poses of the same hierarchical model.

8.7 Cars

There are three types (sedan $4.4 \times 1.78 \times 1.42$ m, SUV and hatchback) in nine paint colours.

Table 9: Parts of a car

Part	Construction
Body	big box, rounded nose (cylinder lying along x, $R_z 90°$), bumpers, underbody
Bonnet	wedge that slopes down towards the front; boot lid box
Cabin	tapered box made of tinted *transparent* glass, so the driver is visible; roof box; SUV uses an upright glass box + roof rails
Pillars	thin boxes stretched between two points by a "beam" helper: the box's y axis is turned onto $\mathbf{b}-\mathbf{a}$ with the rotation $\mathbf{y}\times\mathbf{d}$ by atan2($\lvert\mathbf{y}\times\mathbf{d}\rvert$, $\mathbf{y}\cdot\mathbf{d}$)
Interior	seats, rear bench, dashboard, steering wheel (torus $R_x 70°$)
Lights	head lights (emissive at night + real spot lights), tail and brake lights, reverse lights, blinking orange indicators
Wheels (4)	dark wheel well, tyre cylinder $(2R, .22, 2R)$ $R_z 90°$, chrome rim, 3 spokes, hub; node $R_y(steer)$ for the front wheels, then $R_x(spin)$
Driver door	child node $T(hinge)\cdot R_y(\theta_{door})$; it opens to 70° when the NPC gets in or out

The wheels turn with θ_{spin} += $\Delta s/R$, where Δs is the distance driven. The steering angle follows the car's turning rate, $\delta = \arctan(L_{wb}\,\Delta\psi/\Delta s)$, where L_{wb} is the wheel base and $\Delta\psi$ is the change of heading.

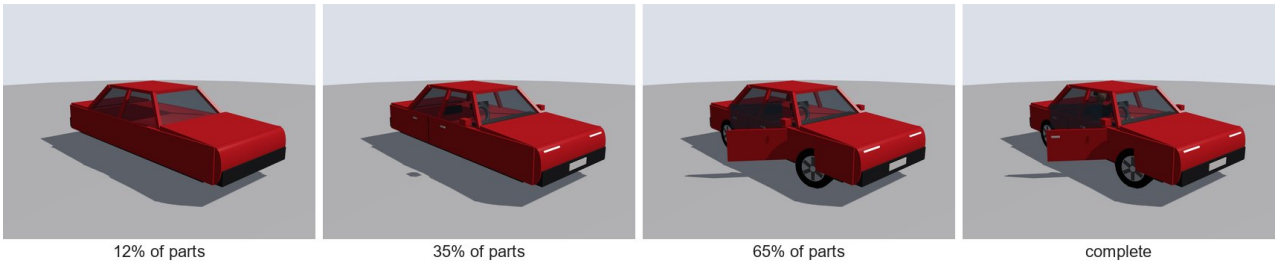
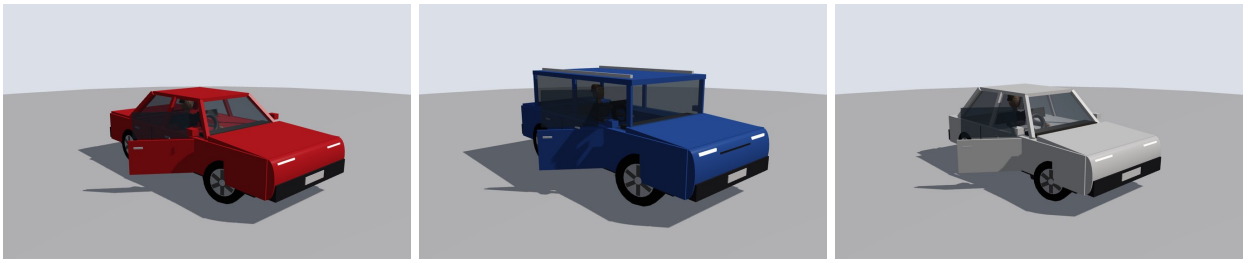


Figure 16: Sedan build stages (the driver is a person model sitting in the drive pose).



(a) Sedan, door open **(b)** SUV **(c)** Hatchback

Figure 17: The three car types.

8.8 The building

Walls are made by one function, `buildWall()`. It takes a straight wall and a list of openings (doors and windows), and splits the wall into pieces (Figure 18):

1. full-height pieces between the openings,
2. for each opening, a piece *below* it (a window sill wall) and a piece *above* it (lintel),
3. for windows: a transparent glass pane, an aluminium frame (4 bars + a middle bar) and a marble sill,
4. for doors: frame trims.

Each piece has two half-thickness layers. The outside layer uses *brick*, and the inside layer uses the room's *plaster* colour (beige lounge, light blue ATM room, sage green library). Wooden skirting boards run along the inside floor line. Floors are tiles (lounge), carpet (ATM room) and wood (library). The ceilings and roof (with parapet, AC units and the sign) are drawn per frame, so the roof can be hidden in the cut-away view. The automatic sliding glass doors slide sideways by $x = \pm(0.6 + 1.15 \cdot \mathit{open})$ while someone stands within 2.2×2.8 m of them. While they are closed they also block the player.

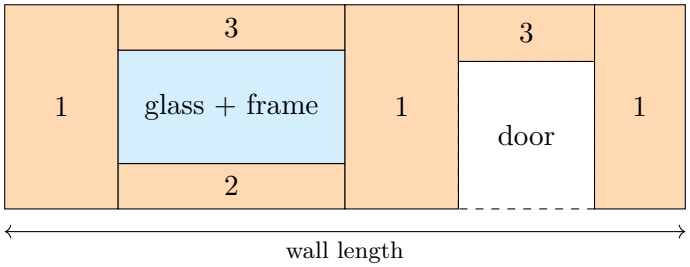


Figure 18: A wall with one window and one door: (1) full pieces, (2) below the window, (3) above the opening.

8.9 The jungle

Tree species.

- *Rainforest tree*: low-poly cylinder trunk (6.5–9 m), 4 **buttress roots** (wedges rotated $R_y(90°i)$), 3 branches (cylinders in nodes $R_y(yaw)R_x(45–60°)$), and an irregular crown of 6–8 foliage blobs (scaled low spheres with a leaf texture), each covered by 2 alpha-tested **leaf cards**. Some branches carry hanging vines.
- *Coconut palm*: a chain of 7 trunk segments. Each segment is a child of the previous one with an extra $R_x(3–7°)$, so the trunk bends smoothly. 9 fronds, each made of two quads (the second is a child of the first and droops by another 30–45°), and 4 coconuts.
- *Emergent giant*: a 14–18 m trunk, 5 large buttresses, 4 branches and a wide layered crown.
- *Bamboo clump*: 9–15 tilted culms (thin cylinders) with node rings and leaf cards.
- *Banana plant*: 1–3 pseudo-stems with 5–8 long leaves (spheres flattened to $(0.55, 0.04, 1.9)$), sometimes with a banana bunch.
- *Undergrowth*: bushes (blobs + leaf cards, some with flowers), ferns (5 arched frond quads + a cross card), grass tufts and flower cards, rocks (1–3 rotated, squashed low spheres with a rock texture) and fallen logs (horizontal cylinder + moss blobs + mushrooms made of a cylinder and a hemisphere).



Figure 19: Rainforest tree build stages: trunk, buttress roots, branches, then crown blobs and leaf cards.

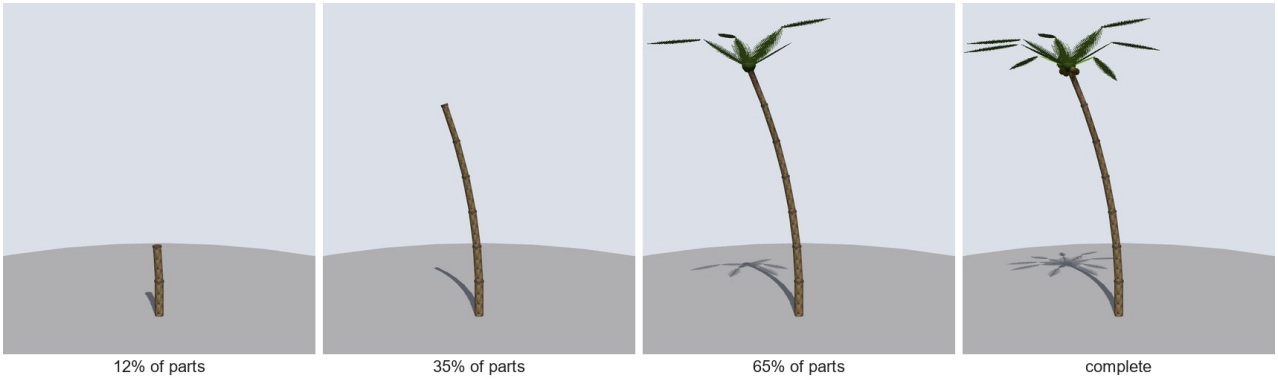
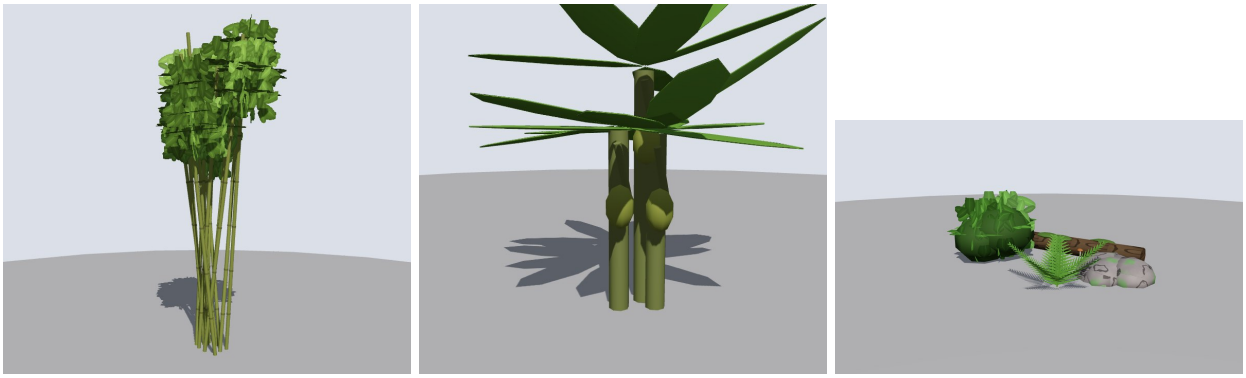


Figure 20: Palm build stages: the chained trunk segments bend a little more at each joint.



(a) Bamboo **(b)** Banana plant **(c)** Bush, fern, rock, log

Figure 21: More plants.

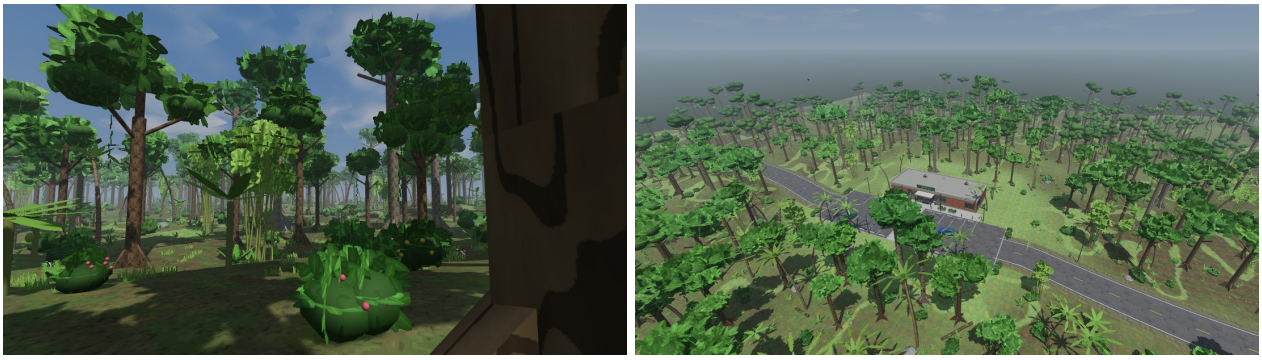
Placement. The world (280×280 m) is covered with a *jittered grid* of 6.6 m cells. Each cell gets one random point. Points inside the cleared area (building, lot and road with a margin) are skipped. A random number then picks the species: 40% rainforest tree, 18% palm, 7% giant, 8% bamboo, 11% banana, 8% bush, and 8% nothing. Every cell also gets 1–5 ferns or bushes and 4–12 grass or flower tufts (more at the edge of the clearing). There is a 12% chance of a rock and a 5% chance of a log. In total the jungle has 797 broad-leaf trees, 296 palms, 120 bamboo clumps, 1,450 bushes, 2,232 ferns, 4,758 grass tufts, 193 rocks and 78 logs. Every trunk also gets a circle collider.

Terrain. The ground is one displaced grid. It is flat in the clearing and along the road, and becomes gently hilly further away:

$$H(x,z) = 14\,\big(\mathrm{fbm}(0.018x,\,0.018z) - 0.45\big) + 1.2\,\mathrm{fbm}(0.07x,\,0.07z)$$

$$h(x,z) = \max\Big(-1.5,\;\; H(x,z)\cdot \mathrm{smoothstep}(0,30,d)\cdot \mathrm{smoothstep}\big(5,14,|z - z_{road}(x)|\big)\Big)$$

Here d is the distance outside the clearing. The road is straight through the lot ($z = 16$) and curves in the jungle with $z_{road}(x) = 16 + 7\sin\big(0.035(|x| - 30)\big)\,\mathrm{sign}(x)$.



(a) Inside the jungle **(b)** From above

Figure 22: The procedural jungle.

9 Lighting

9.1 Lighting model

All lighting is computed per pixel in the fragment shader with the Blinn–Phong model. For one light with direction $\mathbf{L}$, normal $\mathbf{N}$, view direction $\mathbf{V}$ and half vector $\mathbf{H} = \mathrm{normalize}(\mathbf{L} + \mathbf{V})$:

$$C = \underbrace{\mathrm{mix}(A_{ground}, A_{sky}, \tfrac{N_y+1}{2})\,k_d}_{\text{ambient}} + \sum_{lights} \Big(k_d \max(\mathbf{N}\cdot\mathbf{L}, 0) + k_s(\mathbf{N}\cdot\mathbf{H})^n\Big)\, I\, a(d)\, c\, s + E.$$

Here k_d is the surface colour (times the texture), k_s is the specular strength, n is the shininess, I is the light colour times its intensity, $a(d)$ is the attenuation, c is the spot cone factor, s is the shadow factor and E is the emission (screens, LEDs, bulbs). The ambient part uses *hemisphere* lighting: surfaces facing up get the sky colour and surfaces facing down get the ground colour. Inside the building the ambient part is multiplied by 0.45, because the roof blocks the sky. Leaves and grass use soft "wrap" lighting, $0.6\,\mathbf{N}\cdot\mathbf{L} + 0.4$, and are lit from both sides.

Attenuation uses a smooth window, so every light really ends at its range r and can be skipped beyond it:

$$a(d) = \frac{\left[\text{clamp}(1-(d/r)^4,\,0,\,1)\right]^2}{1+d^2}, \qquad c = \text{smoothstep}\big(\cos\theta_{out},\ \cos\theta_{in},\ -\mathbf{L}\cdot\mathbf{D}\big) \text{ for spot lights.}$$

9.2 Light types

Table 10: Light types in the scene

Type	Where
Directional	the **sun** by day and the **moon** by night (one light that follows the clock)
Point	lounge pendant bulbs (4), library pendants (3), ATM tube lights (2), vending machine interiors (3), ATM screen, aquarium hood, TV glow (flickers with the picture), 8 parking lamps and road lamps
Spot	recessed downlights over the vending machines, TV wall and coffee counter, the reading floor lamp, porch lights, **car headlights** (moving spot lights)
Emissive	every lamp model has a glowing bulb or tube when it is on, plus LEDs, screens, signs, fireflies and the marquee
Ambient	hemisphere sky / ground light that changes with the time of day

Every light belongs to a *group*: lounge, spots, ATM room, library, outdoor or machines. Groups can be switched with the F1–F4 keys or with the real **light switches** on the walls, which you can look at and press. The lamp models check their group, so a bulb only glows when its light is on. The outdoor group switches on automatically at dusk.

9.3 Choosing lights every frame

The shader accepts up to 16 point and 8 spot lights. Every frame the light manager:

1. drops lights that are switched off, too far away ($>$ 45 m beyond their range), or whose sphere of influence is outside the camera frustum,
2. keeps the closest ones,
3. sorts them by **region**: ATM room, lounge, library or outdoors.

Each pixel works out its own region from its position and loops *only* over the lights of that region. This has two benefits. Light no longer leaks through walls into the next room, and pixels do much less work. In the lounge view this change raised the frame rate from 53 to 63 FPS.



(a) Night, lamps on



(b) Night, lamps switched off (only machine glow and moonlight)

Figure 23: The same room with and without its bulbs and spot lights.



(a) Library at night: warm pendants and a reading lamp



(b) Sodium street lamps and car lights

Figure 24: Different kinds of light at night.

10 Day and Night

A 24-hour clock runs at 1 game hour per 20 s by default (keys change the speed, pause it, or jump between day and night). The sun's direction comes from the hour:

$$a = \frac{hour - 6}{12}\pi, \qquad \mathbf{s} = \text{normalize}(0.95\cos a,\ \sin a,\ 0.42), \qquad \mathbf{m} = \text{normalize}(-0.8\,s_x,\ -s_y,\ 0.3).$$

So the sun rises in the east ($+x$) at 6:00, is highest at noon and sets at 18:00, and the moon is on the opposite side. All other values are blended from the sun height s_y: the sky zenith and horizon colours (with an orange glow near sunrise and sunset), the sun colour (warm when low), the ambient light, the fog colour and density, the cloud colour and the exposure. The night factor $1 - \text{smoothstep}(-0.14, 0.10, s_y)$ shows the stars and moon, turns on the street lamps and car headlights, and wakes up the fireflies.



(a) Sunrise 6:18 **(b)** Day 10:30

(c) Sunset 17:36 **(d)** Night 21:00

Figure 25: The same view at four times of day.

11 Textures

11.1 Procedural textures

All 25 textures are created at start-up from *value noise*. Random values are placed on a grid and smoothly interpolated between grid points. Several octaves are then added together (fractal Brownian motion):

$$\mathrm{fbm}(\mathbf{p}) = \frac{\sum_{i=0}^{k-1} 0.5^i \,\mathrm{noise}(2^i \mathbf{p})}{\sum_{i=0}^{k-1} 0.5^i}.$$

The noise wraps around at the texture border, so the textures tile without seams. Patterns are built on top of it: rows of bricks with mortar, wooden planks with grain $\sin(6v + \mathrm{noise})$, tiles with grout, marble veins, woven fabric, bark ridges, and also *cut-out* textures (leaves, grass blades, fern, palm frond, flowers, vine) whose alpha channel marks where the leaf is.

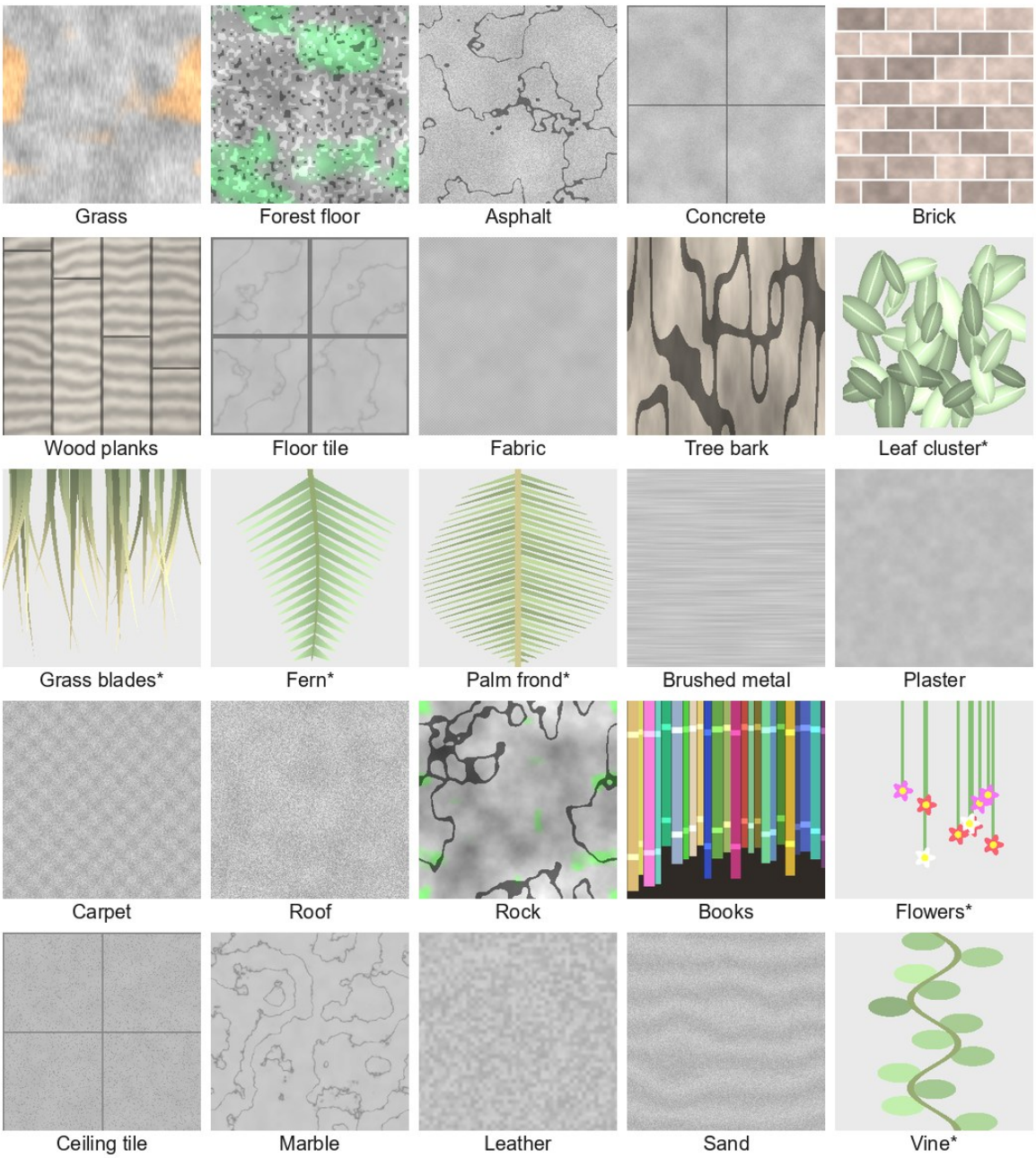


Figure 26: All procedural textures ($*$ = alpha cut-out, light grey shows the transparent part).

11.2 How textures are applied

- All layers live in one `GL_TEXTURE_2D_ARRAY` (256×256, mip-maps, $4\times$ anisotropic filtering). Each instance only stores a layer number, so a texture change never breaks an instanced draw call.
- A texture stores a *detail* value. The final colour is $k_d = \text{colour} \times 2 \cdot \text{texel}$, so the same brick texture works for any brick colour.
- **Object-space box mapping**: the shader picks the plane of the largest normal component and uses the local position multiplied by the object's scale: $(u, v) = (x, z)$, (z, y) or (x, y) times the repeats per metre. A 10 m wall therefore shows ten times more bricks than a 1 m box, without any UV work.
- Cut-out cards use their mesh UVs, and pixels with $\alpha < 0.5$ are discarded (also in the shadow pass, so leaves cast leaf-shaped shadows).

- The terrain mixes two textures (lawn grass and forest floor) using a large-scale noise and the distance from the clearing.

12 Shadows

Shadows use **shadow mapping** (Figure 27).

1. **Depth pass**: the scene is drawn from the sun with an orthographic camera (±42 m box around the player, 2048×2048 depth texture). Only depth is stored.
2. **Main pass**: every pixel is moved into the light's space, $\mathbf{p}_L = P_L V_L\,(\mathbf{p} + 0.05\,\mathbf{N})$. If its depth is larger than the stored depth, it is in shadow.

To make the shadows look good:

- a slope-scaled bias $0.0006 + 0.0012(1 - \mathbf{N}\cdot\mathbf{L})$, a 5 cm normal offset and a polygon offset in the depth pass remove "shadow acne",
- **PCF**: 5 hardware-compared samples (the centre counts twice) give soft edges,
- the light box moves in whole texel steps, so shadows do not shimmer when the player walks,
- glass, grass and other tiny details do not cast shadows, which saves time.

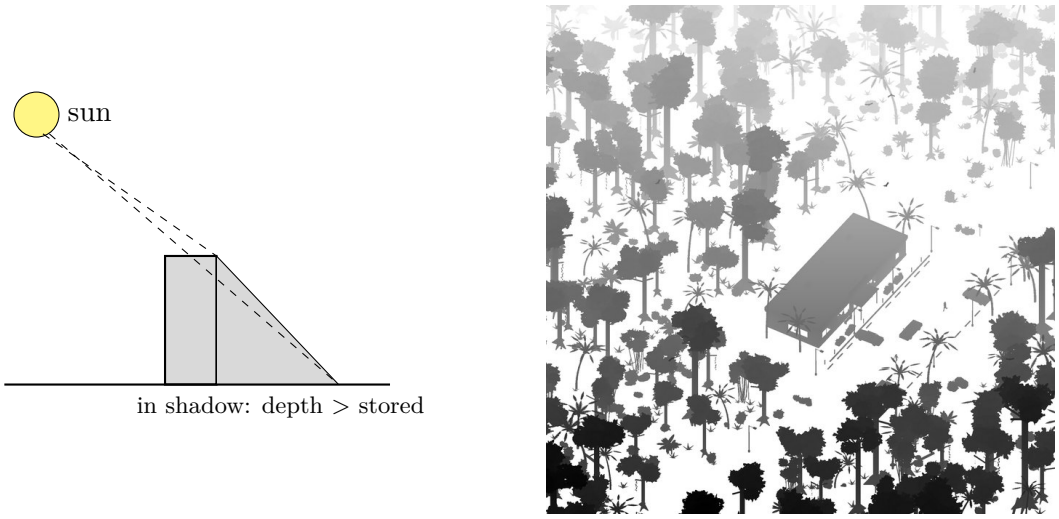


Figure 27: Left: the idea of shadow mapping. Right: the real depth image seen by the sun (darker = closer to the sun). The building, cars and every tree crown are visible.



(a) Shadows on **(b)** Shadows off

Figure 28: Shadows give the scene depth and show where objects touch the ground.

13 Sky, Fog, Tone Mapping and Wind

- **Sky**: one full-screen triangle. For each pixel the view ray is rebuilt from the inverse view-projection matrix. The colour is $\text{mix}(\mathit{horizon}, \mathit{zenith}, h^{0.55})$, plus a sun disc $(\max(\mathbf{d} \cdot \mathbf{s}, 0)^{900})$ with a glow, a moon, twinkling stars (a hash on a 3D grid) and moving clouds (4-octave noise projected on a sky plane).
- **Fog**: $f = 1 - e^{-(d\rho)^{1.5}}$ blends to the horizon colour. It hides the end of the world and gives a humid jungle look. The density is higher at night and lower when the camera is high up.
- **Tone mapping**: $C' = 1 - e^{-C \cdot \mathit{exposure}}$ keeps bright lamps from turning flat white. The exposure rises at night, like an eye adapting to the dark.
- **Wind**: leaves and grass sway in the vertex shader. The offset grows with the height inside the shape: $\Delta x = 0.05\, h\, \min(s_y, 2.5) \sin(1.7t + 0.35x + 0.27z)$. Aquarium plants reuse this as a water current.

14 Basic Mode versus Enhanced Mode

Press **B** (or click the HUD button) to switch modes. **T**, **G** and **F** switch textures, shadows and fog one at a time.

Feature	Basic	Enhanced
Basic shapes, transformations, all objects	yes	yes
Blinn–Phong lighting with all lights	yes	yes
Procedural textures	–	yes
Shadow mapping (PCF)	–	yes
Fog	–	yes
Sky with sun, clouds, stars	simple gradient	yes
Wind animation	–	yes
Tone mapping	clamp	exponential



(a) Basic



(b) Enhanced



(c) Basic (interior)



(d) Enhanced without textures

Figure 29: Basic and Enhanced rendering.

15 Interaction: Using the Machines

15.1 Look-and-press picking

Every button, slot, flap, switch, TV or aquarium registers an *oriented box* (its matrix M and half size $\mathbf{h}$) every frame. A ray goes from the camera through the centre of the screen. For each box the ray is moved into the box's own space with M^{-1}, and the *slab test* is applied on each axis:

$$t_1 = \frac{-h_a - o_a}{d_a}, \quad t_2 = \frac{h_a - o_a}{d_a}, \qquad t_{near} = \max_a \min(t_1, t_2), \quad t_{far} = \min_a \max(t_1, t_2).$$

The ray hits the box if $t_{near} \le t_{far}$ and t_{near} is within reach (2.4 m). The closest box wins. Its prompt is shown under the crosshair (for example "Insert Tk 10 coin"), and it glows. Pressing **E** or the left mouse button runs its action. As an automatic test, the program aims the camera at fourteen different parts and records what it hits (Listing 1). Every part was hit correctly.

Listing 1: Automatic interaction test (part aimed at → prompt found)

```
vending coin slot -> [Insert Tk 10 coin (you have Tk 1120)]
vending note slot -> [Insert Tk 50 note (you have Tk 1120)]
vending key B -> [Press [B]]
vending key 2 -> [Press [2]]
vending OK -> [Press [OK]]
vending flap -> [Collection flap]
ATM card slot -> [Insert bank card]
ATM side key L1 -> [Side key L1]
ATM keypad 5 -> [Keypad [5]]
ATM cash slot -> [Cash dispenser]
coffee button 1 -> [Make Espresso (free)]
TV -> [Change TV channel (now: News)]
aquarium -> [Feed the fish]
light switch -> [Turn OFF lounge lights]
```

15.2 ATM: step by step

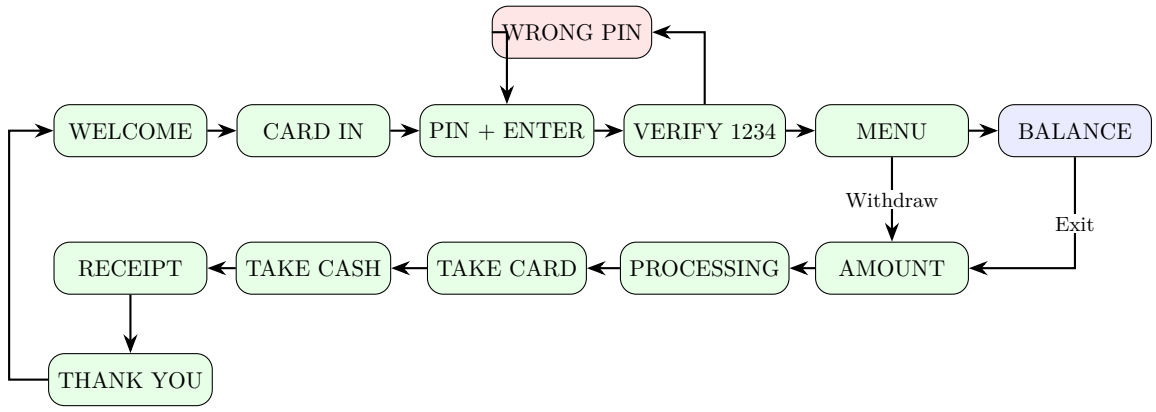
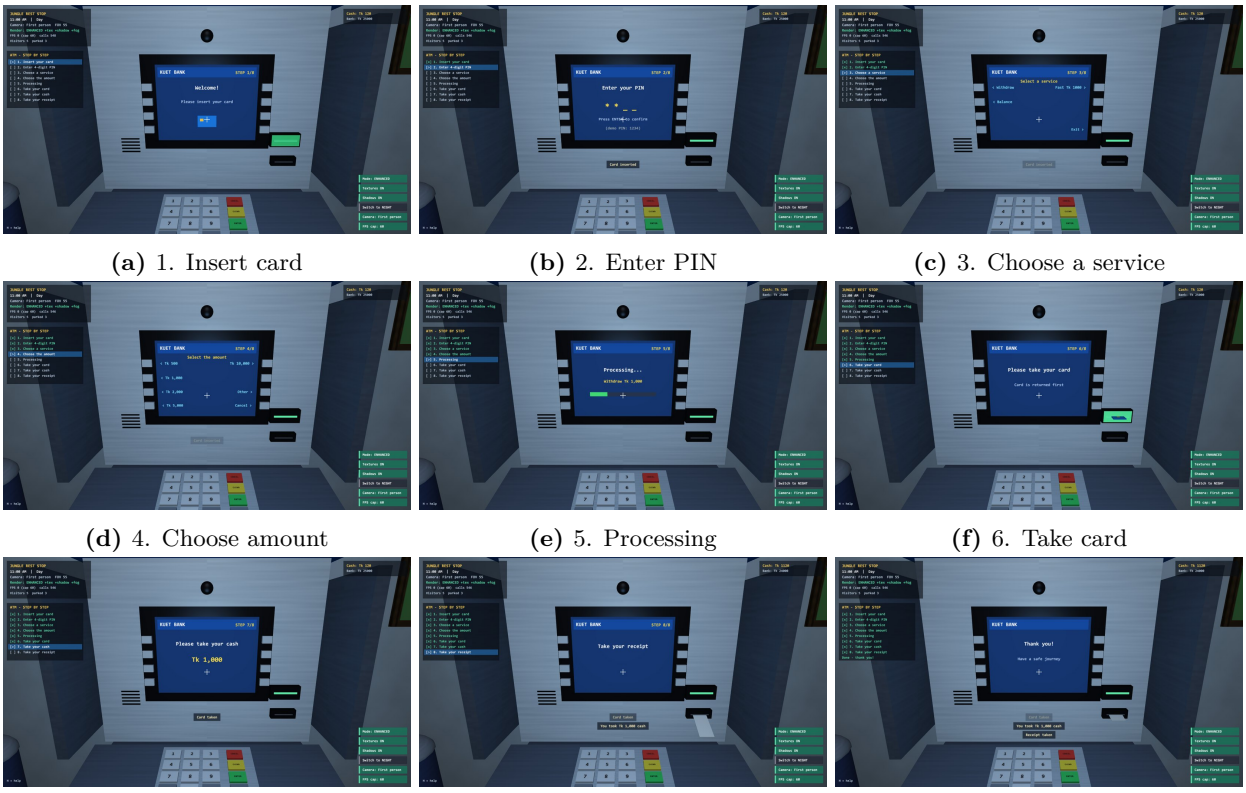


Figure 30: ATM state machine. CANCEL at any step jumps to TAKE CARD. The card is always returned before the cash, like a real ATM.

The same eight steps are listed in a HUD panel, and the current step is highlighted. Inside the machine, the card slides in and out, the PIN appears as stars, the bills slide out of the cash slot and the receipt paper curls out of the printer. Taking the cash adds Taka to the player's wallet and takes it from the bank balance. That money can then be spent at the vending machines.



(a) 1. Insert card **(b)** 2. Enter PIN **(c)** 3. Choose a service

(d) 4. Choose amount **(e)** 5. Processing **(f)** 6. Take card

(g) 7. Take cash **(h)** 8. Take receipt **(i)** Done

Figure 31: A full withdrawal of Tk 1,000, with the step list on the left.

15.3 Coffee machine and TV



(a) Brewing a cappuccino (stream, rising coffee, steam) **(b)** Ready, take the cup

Figure 32: Coffee machine: `IDLE` → `CUP_DROP` → `GRINDING` → `BREWING` → `READY`.



(a) News + ticker **(b)** Nature **(c)** Live football **(d)** Weather (game time)

Figure 33: The four animated TV channels.

16 NPC Visitors

16.1 Life of a visitor

Visitors can **only** appear inside a car at the far end of the road. A new car comes every 8–18 s if there is a free parking spot and the road entrance is clear. At most six visitors are active at once.

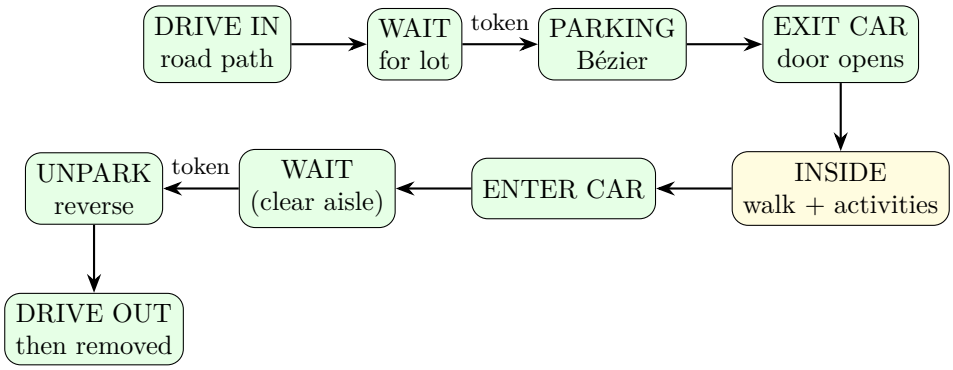


Figure 34: Visitor state machine.

16.2 Driving and parking

Cars follow *paths*: lists of points with the running length s. The car's position comes from interpolation at s, and its heading from the direction of the segment. The road path samples $z_{road}(x)$ every 3 m. Parking uses a cubic Bézier curve

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\,\mathbf{P}_1 + 3(1-t)t^2\,\mathbf{P}_2 + t^3\mathbf{P}_3,$$

with $\mathbf{P}_0 = (x_s{+}5, 16)$ and $\mathbf{P}_1 = (x_s{+}2.2, 16)$ along the aisle, and $\mathbf{P}_2 = (x_s, z_s{\pm}2.8)$ and $\mathbf{P}_3 = (x_s, z_s)$ in the spot. The tangents at both ends therefore match the aisle and the spot, so the turn is smooth.

Leaving uses a second curve driven in *reverse* (the heading is the motion direction +180°, the reverse lights are on), and then the car drives forward to the west end of the road.

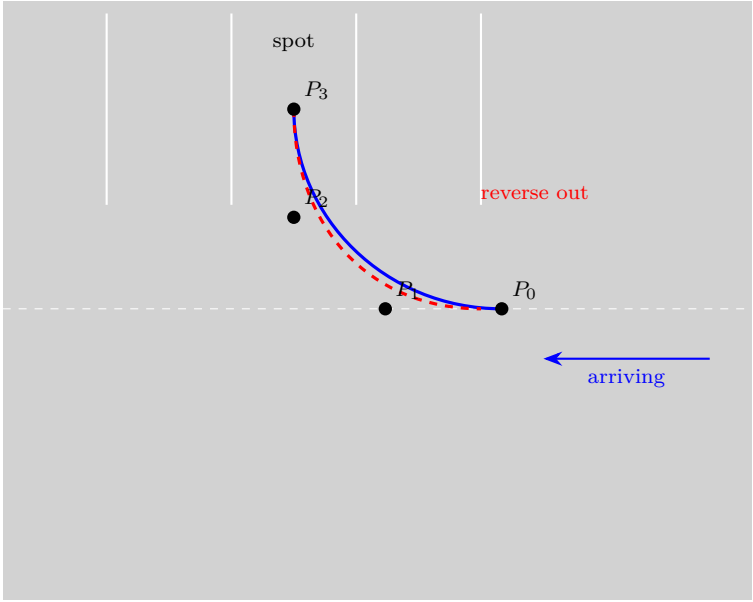


Figure 35: Parking (blue, forward) and leaving (red, reverse) curves for one spot.

Speed is limited by the place (9 m/s on the road, 4 m/s in the lot, 1.8 m/s while parking, 1.3 m/s while reversing) and by the distance s_{left} still to go: $v \leq \sqrt{2 \cdot 2.2 \cdot s_{left}} + 0.25$, so cars brake smoothly. A car stops if another car is less than 7.5 m ahead in its lane, or if a person (NPC or player) is less than 5.5 m ahead. Only one car may park or unpark at a time: it must hold the "lot token". Cars in the aisle wait while the token holder moves in front of them, and a car only starts to reverse when the aisle behind it is clear. These two rules prevent deadlocks.

16.3 Walking and activities

Inside the building, people walk on a waypoint graph of 18 nodes and 22 edges (doorways, room hubs, the queue entrance at the ATM, and so on). The shortest route is found with **Dijkstra's algorithm**, and the route to the car is added at the end. Walking speed is 1.35 m/s, and people wait if the player blocks them. Every visitor has a plan of 1–3 activities. Each one *reserves* its resource first (a machine, a seat, a reading spot), so two people never use the same thing:

- **Vending**: walk to the machine, press the keys (press pose), wait for the drop, bend down, take the item and carry it. This really runs the machine's state machine.
- **Coffee**: choose a drink, wait, take the cup and carry it.
- **ATM**: run a complete ATM session. The ATM "presses its own buttons" with small pauses, so the screen shows every step.
- **Sit and watch TV** (7 sofa seats) or **read** (2 library armchairs) for 12–28 s.
- **Browse books** or **watch the fish**.

Listing 2 shows the beginning of a real trace recorded by the program.

Listing 2: Start of a visitor trace (simulation time, visitor, spot, what it is doing)

```
t= 4.0s visitor 0 (spot 18): driving in
t= 8.7s visitor 1 (spot 10): driving in
t= 14.2s visitor 2 (spot 13): driving in
t= 19.7s visitor 0 (spot 18): waiting to park
t= 19.7s visitor 0 (spot 18): parking
```

```
t= 22.2s visitor 3 (spot 3): driving in
t= 24.5s visitor 0 (spot 18): getting out
t= 26.8s visitor 0 (spot 18): walking in
t= 27.3s visitor 4 (spot 0): driving in
t= 30.9s visitor 1 (spot 10): waiting to park
t= 30.9s visitor 1 (spot 10): parking
t= 32.9s visitor 5 (spot 14): driving in
t= 35.7s visitor 1 (spot 10): getting out
t= 38.0s visitor 1 (spot 10): walking in
t= 43.4s visitor 2 (spot 13): waiting to park
t= 43.4s visitor 2 (spot 13): parking
t= 47.4s visitor 0 (spot 18): browsing books
t= 48.2s visitor 2 (spot 13): getting out
t= 50.5s visitor 2 (spot 13): walking in
t= 51.1s visitor 3 (spot 3): waiting to park
t= 51.2s visitor 3 (spot 3): parking
t= 51.4s visitor 4 (spot 0): waiting to park
t= 54.1s visitor 1 (spot 10): watching fish
t= 56.0s visitor 3 (spot 3): getting out
t= 56.0s visitor 4 (spot 0): parking
t= 58.2s visitor 3 (spot 3): walking in
t= 60.8s visitor 4 (spot 0): getting out
t= 63.0s visitor 4 (spot 0): walking in
t= 69.5s visitor 5 (spot 14): waiting to park
t= 69.6s visitor 5 (spot 14): parking
t= 69.8s visitor 0 (spot 18): leaving
t= 73.3s visitor 2 (spot 13): at a vending machine
t= 73.9s visitor 1 (spot 10): leaving
t= 74.4s visitor 5 (spot 14): getting out
```



(a) Cars parked, a visitor walking to the entrance



(b) Visitors inside (TV on)

Figure 36: NPC visitors.

17 The Player and the Cameras

- **First person**: the camera is at eye height (1.62 m), with mouse look and a small head bob $0.028 \sin(2\varphi)$ while walking.
- **Third person**: the player's own model is visible. The camera sits 3.4 m behind the head. A ray from the head to the camera is tested against the walls, and the camera moves closer if a wall is in the way.
- **Free camera**: fly anywhere (W A S D, Space and Ctrl) to inspect the scene, for example with the roof cut-away view.
- **FOV**: the mouse wheel or $+/-$ changes the vertical field of view from 25° to 110°. This changes the $1/\tan(f/2)$ terms of the projection matrix.

Collision. The player is a circle with a radius of 0.3 m. For every obstacle box (walls, furniture, machines, cars), the circle centre is moved into the box's space, the closest point $\mathbf{q} = \text{clamp}(\mathbf{p}, -\mathbf{h}, \mathbf{h})$ is found, and if $|\mathbf{p} - \mathbf{q}| < r$ the circle is pushed out along $\mathbf{p} - \mathbf{q}$ (three passes). Tree trunks and people are circles. Gravity and jumping use $v_y \mathrel{-}= 12\,\Delta t$ on top of the terrain height.



(a) Third person



(b) Free camera, roof cut-away, night



(c) FOV 35°



(d) FOV 70°



(e) FOV 100°

Figure 37: Camera modes and field of view.

18 Performance on a Weak GPU

18.1 Techniques

- **Instanced drawing**: all copies of one basic shape are drawn with one `glDrawElementsInstanced` call. Each instance carries 37 floats (model matrix 16, normal matrix 9, colour and alpha 4, emission and texture layer 4, specular, shininess, texture scale and flags 4).
- **Static world in chunks**: 82,414 shapes that never move are collected once (in 0.44 s) into 101 chunks of 32×32 m, each with its own GPU buffer.
- **Culling**: a chunk is skipped if its bounding box is outside the view frustum (6 planes taken from the view-projection matrix) or more than 165 m away. Small things have shorter ranges: grass and flowers 45 m, and bushes, ferns and leaf cards 85 m.
- **Shadow pass** only draws chunks inside the light's box, and skips grass and glass.
- **Per-room light lists** and frustum culling of lights (see the Lighting section).
- **Moving objects** are written into streaming buffers once per frame. Transparent objects are sorted from back to front.
- **Frame cap**: after each frame the program sleeps until $1/cap - 1.5$ ms and then waits the last moment in a loop (with a 1 ms Windows timer resolution). This gives a very steady 60 or 30 FPS and keeps the laptop cool.
- **Low mode** (`-low`): a 1024 shadow map, fewer plants, fewer visitors and no MSAA.

18.2 Measured speed

The program measures itself (`-shots` mode, with no frame cap, 60 frames per view, 1280×720, 4×MSAA, Intel Iris Xe). The triangle count includes the shadow pass.

Table 11: Frame time without the cap

View	Enhanced		Basic		Triangles
	ms	FPS	ms	FPS	(enh.)
Outside, day	13.7	73	9.2	109	1.81M
Main lounge	15.8	63	12.5	80	1.90M
Inside the jungle	11.1	90	7.2	138	1.55M
Parking lot, night	15.9	63	15.1	66	1.01M
Aerial view	13.9	72	10.2	98	1.74M

Every view runs faster than 60 FPS even in Enhanced mode, so the 60 FPS cap always holds. During development, three optimisations made the biggest difference: per-room light lists, larger chunks (fewer draw calls) and frustum culling of lights. Together they raised the slowest view from 37 to over 60 FPS.

19 Controls

Key	Action
W A S D, Shift, Space	move, run, jump
Mouse	look around
E / left click	use the part in the crosshair
V	first person → third person → free camera
Mouse wheel, + / −	change the field of view
B	**Basic ↔ Enhanced**
T / G / F	textures / shadows / fog
N, [ ], M	day ↔ night, time speed, pause time
R	roof cut-away
L	frame cap 60 ↔ 30
K	call a visitor car now
F1–F4	light groups (lounge, spots, ATM + library, outdoor)
F5, F12	hide HUD, save screenshot
Alt	free the mouse to click the HUD buttons
H, Esc	help, quit

Vending machine: insert money, type a code (row letter + column number, for example `B2`), press OK, push the flap, take the change. **ATM:** insert the card, PIN `1234`, use the side keys.

20 More Screenshots



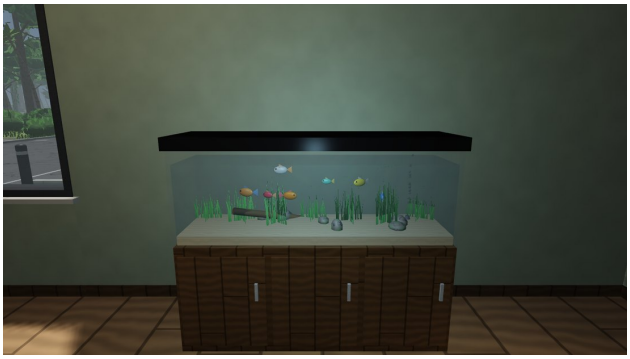
Figure 38: The main lounge by day: sofas, TV wall with wooden slats, coffee counter, three vending machines, the ATM room on the left and the library on the right.



(a) ATM room with queue stanchions



(b) Library



(c) Aquarium

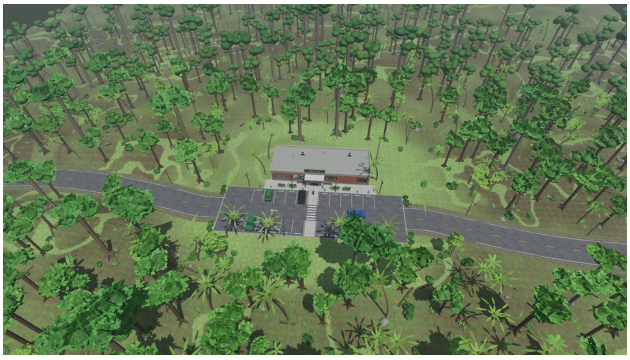


(d) The three vending machines (a visitor is buying)

Figure 39: The three rooms.



(a) Cut-away by day



(b) Aerial view



(c) Road through the jungle



(d) The same road at night

Figure 40: Outside views.

21 Challenges and Solutions

Problem	Solution
Thousands of shapes made the GPU slow	instancing, a static world in chunks, culling and LOD distances
Many lights per pixel were costly	per-room light regions and frustum culling of lights
Light leaked through walls	each room only uses its own lights
Shadow acne and shimmering	slope bias, normal offset, polygon offset, light box snapped to texels
Trees looked like “mushrooms”	irregular crowns made of many blobs plus alpha-tested leaf cards
Text on machine screens	one GDI font atlas, and text quads laid on a canvas in 3D
Cars blocking each other in the lot	one “lot token” for parking or unparking, plus look-ahead braking
The driver’s head went through the roof	the seat height was tuned for each car type
Checking that clicks hit the right button	an automatic aim-and-probe test that writes a log
Report screenshots had to stay up to date	a `-shots` mode renders every figure, table and benchmark

22 Conclusion

The old vending machine scene became a complete small world. Three connected rooms hold working machines, people arrive by car and behave on their own, the jungle around them is dense, and the light changes from morning to night. All of it still follows the main rule of the lab: every object is a set of basic shapes placed with translation, rotation and scaling, and parts are organised in parent and child nodes. The project also covers the main topics of a graphics course in one program: lighting models,

light types, textures, shadow mapping, fog, transparency, animation, picking, collision, cameras and performance. With instancing, culling and smart light handling, it runs at a steady 60 FPS on a laptop with only integrated graphics.

Future work

- Shadows from lamps (cube shadow maps), and screen-space ambient occlusion.
- Sound: footsteps, machine beeps, cars, birds and rain.
- Weather (rain and puddles), and visitors talking to each other.
- Let the player drive a car or sit on the sofas.

References

[1] J. Kessenich, G. Sellers, D. Shreiner. *OpenGL Programming Guide*, 9th ed., Addison-Wesley, 2016.

[2] J. de Vries. *Learn OpenGL*, `https://learnopengl.com` (lighting, shadow mapping, instancing).

[3] D. Hearn, M. P. Baker, W. Carithers. *Computer Graphics with OpenGL*, 4th ed., Pearson, 2010.

[4] T. Akenine-Möller, E. Haines, N. Hoffman et al. *Real-Time Rendering*, 4th ed., CRC Press, 2018.

[5] J. F. Blinn. Models of light reflection for computer synthesized pictures. *SIGGRAPH*, 1977.

[6] L. Williams. Casting curved shadows on curved surfaces. *SIGGRAPH*, 1978.

[7] K. Perlin. An image synthesizer. *SIGGRAPH*, 1985.

[8] GLFW documentation, `https://www.glfw.org`; GLAD loader, `https://glad.dav1d.de`.